Solution

Yes — here is the **complete solution in one cell**, matching the task.

Important: Because we use `window()` + `withWatermark()` with `append` mode, the window aggregation is appropriate for the Parquet output. The city aggregation is also written separately.

```
# ============================================================
# PySpark Structured Streaming - Intermediate E-Commerce Task
# JSON -> Transform -> Window Aggregation -> Parquet + JSON
# ============================================================

from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *


# ============================================================
# 1. Create Spark Session
# ============================================================

spark = (
    SparkSession.builder
    .appName("Intermediate E-Commerce Streaming")
    .getOrCreate()
)

print(f"Spark version: {spark.version}")


# ============================================================
# 2. Define Schema
# ============================================================

schema = StructType([
    StructField("order_id", StringType(), True),
    StructField("customer_id", IntegerType(), True),
    StructField("customer_name", StringType(), True),
    StructField("product", StringType(), True),
    StructField("category", StringType(), True),
    StructField("quantity", IntegerType(), True),
    StructField("price", DoubleType(), True),
    StructField("city", StringType(), True),
    StructField("timestamp", TimestampType(), True)
])


# ============================================================
# 3. Read JSON as Streaming DataFrame
# ============================================================

orders_stream = (
    spark.readStream
    .schema(schema)
```

```
    .json("/data/json/orders.json")
)


# ============================================================
# 4. Transformations
# ============================================================

transformed = (
    orders_stream

    # Keep valid orders
    .filter(
        (col("quantity") > 0) &
        (col("price") > 0)
    )

    # Calculate total amount
    .withColumn(
        "total_amount",
        col("quantity") * col("price")
    )

    # Create customer type
    .withColumn(
        "customer_type",
        when(col("total_amount") >= 1000, "VIP")
        .when(col("total_amount") >= 500, "Premium")
        .otherwise("Regular")
    )

    # Extract hour from timestamp
    .withColumn(
        "order_hour",
        hour(col("timestamp"))
    )

    # Watermark for late data
    .withWatermark(
        "timestamp",
        "10 minutes"
    )
)


# ============================================================
# 5. Window Aggregation
# ============================================================

window_sales = (
    transformed

    .groupBy(
        window(col("timestamp"), "5 minutes"),
        col("category")
    )
```

```
    .agg(
        sum("total_amount").alias("total_sales"),
        sum("quantity").alias("total_quantity"),
        count("order_id").alias("number_of_orders")
    )

    # Make window columns easier to read
    .select(
        col("window.start").alias("window_start"),
        col("window.end").alias("window_end"),
        col("category"),
        col("total_sales"),
        col("total_quantity"),
        col("number_of_orders")
    )
)


# ============================================================
# 6. City-Level Aggregation
# ============================================================

city_sales = (
    transformed

    .groupBy("city")

    .agg(
        sum("total_amount").alias("total_sales"),
        sum("quantity").alias("total_quantity"),
        count("order_id").alias("number_of_orders"),
        avg("total_amount").alias("average_order_value")
    )
)


# ============================================================
# 7. Write Window Aggregation to Parquet
# ============================================================

query_parquet = (
    window_sales
    .writeStream
    .format("parquet")
    .outputMode("append")
    .option("path", "/data/sink_sales")
    .option("checkpointLocation", "/data/sink_sales/checkpoint")
    .trigger(processingTime="10 seconds")
    .start()
)


# ============================================================
# 8. Write City Aggregation to JSON
# ============================================================

query_json = (
```

```
    city_sales
    .writeStream
    .format("json")
    .outputMode("complete")
    .option("path", "/data/sink_city")
    .option("checkpointLocation", "/data/sink_city/checkpoint")
    .trigger(processingTime="10 seconds")
    .start()
)


# ============================================================
# 9. Display Streaming Data
# ============================================================

print("Streaming queries started...")
print("Parquet output: /data/sink_sales")
print("JSON output:    /data/sink_city")


# ============================================================
# 10. Wait for Streaming Queries
# ============================================================

query_parquet.awaitTermination()
query_json.awaitTermination()
```

One correction from the original task

For the **city aggregation**, I changed:

```
.outputMode("append")
```

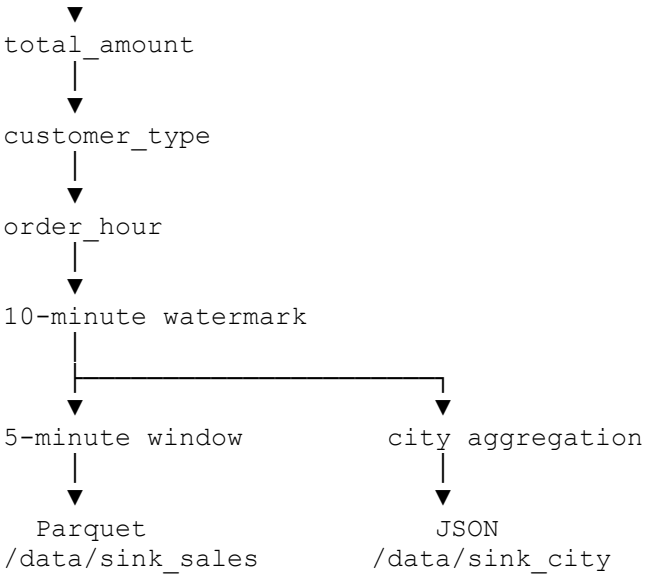
to:

```
.outputMode("complete")
```

because a normal `groupBy("city")` aggregation continuously updates the city totals. **Complete mode** writes the current complete aggregation table each trigger.

So the final architecture is:

```
orders.json
    |
    ▼
readStream
    |
    ▼
Schema
    |
    ▼
Filter invalid orders
    |
```



To test it, add another JSON file containing new orders into:

```
/data/json/
```

For example:

```
hdfs dfs -put new_orders.json /data/json/
```

Spark Structured Streaming will detect the new file and process it.